

Текст выступления по слайдам

Слайд 1. Semantic News Novelty

Идея проекта началась с простой личной боли: я устал от потока похожих новостей. Одно и то же событие пересказывается десятками публикаций, заголовки отличаются, а смысл часто остаётся тем же.

Semantic News Novelty — это сервис для снижения инфошума в новостном потоке. Он помогает превратить поток отдельных публикаций в понятные сюжеты и быстрее увидеть, где действительно появилось что-то новое.

Слайд 2. Проблема

Для обычного читателя повторяющиеся новости — это раздражение. Для аналитика это уже рабочая проблема.

Когда аналитик готовит обзор по большому новостному потоку, ему нужно быстро увидеть основные сюжеты и не пропустить в них главное. Если этого не сделать, часы уходят на ручную вычитку, а важное обновление можно просто не заметить среди пересказов.

Слайд 3. Целевая аудитория

Непосредственный пользователь продукта — аналитик или редактор обзора. Именно он работает с новостным потоком, готовит обзоры, отслеживает сюжеты и обновления.

Заказчик — аналитическое агентство или подразделение. Для него ценность в том, чтобы быстрее выпускать обзоры и снижать ручную нагрузку команды.

Я не делаю основным фокусом крупные корпорации, которые могут купить дорогое комплексное решение или разработать узкую систему внутри. Фокус проекта — малые и средние аналитические команды без собственной ML-разработки.

Слайд 4. Рынок и альтернативы

Я рассматриваю рынок через альтернативы, с которыми реально конкурирует такой инструмент.

Самый простой вариант — ручная вычитка. Она даёт контроль качества, но плохо масштабируется и быстро превращается в рутину.

RSS, Telegram и поисковики помогают быстро собрать поток, но не дают структуру сюжетов и не выделяют смысловую новизну.

Большие full-cycle SaaS закрывают широкий мониторинг, но для части команд могут быть дорогими или избыточными.

Semantic News Novelty занимает более узкую позицию: мы не заменяем сбор новостей, а превращаем уже собранный поток в рабочий материал — с сюжетами, новизной и поиском по смыслу.

Слайд 5. Продукт

Работа продукта максимально простая: он группирует новости и показывает новое.

Сервис берёт поток публикаций, собирает похожие новости в сюжеты, оценивает смысловую новизну внутри сюжета и позволяет искать связанные материалы по смыслу.

На выходе аналитик получает не плоскую ленту публикаций, а структуру для обзора: сюжеты, новизну и поиск по смыслу.

Важно: это не замена аналитика. Редактор может проверить результат модели и исправить метки. Идея в том, чтобы убрать рутину и дать человеку более удобную структуру для анализа.

Слайд 6. MVP: от модели к рабочему сервису

В MVP уже есть основные продуктовые сценарии.

Пользователь может искать новости по смыслу и по датам, а также видеть историю поиска.

Редактор может импортировать CSV, работать с черновиками и вручную корректировать результат модели.

Администратор управляет пользователями, ролями, организациями, балансом и кредитами.

Под капотом это уже не просто ноутбук с моделью, а сервис с API, базой данных, очередью задач, модельным воркером и мониторингом. Технологии на слайде указаны мелко, потому что главное здесь — не стек сам по себе, а то, что MVP уже поддерживает рабочие сценарии.

Слайд 7. Метрики: польза для аналитика

Метрики я выбирал не просто как стандартный набор F1 и recall, а под пользовательскую задачу аналитика.

Для аналитика важны три вещи.

Первая — меньше повторов в выдаче. В Top-10 стало на 75% меньше повторяющихся публикаций, а MRR@10 вырос с 0.91 до 0.99.

Вторая — меньше риск пропустить значимое обновление. Recall по значимым обновлениям вырос с 0.51 до 0.99, а F1 — с 0.63 до 0.91.

Третья — лучше собранные сюжеты. Пайплайн находит 83% правильных связей внутри сюжетов, а F1 группировки вырос с 0.77 до 0.90.

Все эти значения сравниваются с моим собственным бейзлайном. Улучшения получены за счёт финального пайплайна: BGE-M3 эмбедингов, настройки кластеризации и модели значимости.

Слайд 8. Схема сервиса

Архитектура построена как сервис, а не как одиночный скрипт.

Пользователь работает через UI. Запросы идут в API. Тяжёлая обработка новостей уходит в очередь задач, чтобы интерфейс не блокировался.

ML-воркер считает эмбединги, собирает новости в кластеры и определяет новизну. Данные и векторы хранятся в PostgreSQL с pgvector.

Отдельно есть мониторинг через Prometheus и Grafana: можно видеть очередь, скорость обработки, ошибки и потребление ресурсов.

Такая схема позволяет масштабировать обработку: например, запускать несколько воркеров и использовать CPU или GPU-режимы.

Слайд 9. Применение AI-инструментов

AI-инструменты я использовал как формат совместной работы.

Они помогали обсуждать архитектуру, находить слабые места, разбирать ошибки и ускорять рутинные задачи: рефакторинг, документацию, подготовку вариантов формулировок.

Но AI не выбирал за меня метрики, не проверял качество вместо меня и не принимал финальные решения по архитектуре и продукту.

Мой принцип такой: AI — помощник в разработке и thinking partner, но не автор проекта.

Слайд 10. Демо

В демо я не буду показывать весь продукт. Я покажу один основной сценарий.

Сначала редактор загружает заранее подготовленный CSV с новостями. Затем новости попадают в черновики, отправляются в модель и получают результат обработки.

После этого я покажу, как можно перейти к сюжетам или найти связанные материалы по смыслу.

Я буду комментировать каждый шаг по ходу. Типы аккаунтов, админка и Grafana тоже есть в MVP, но в коротком питче они не должны становиться центром демо.

Слайд 11. Преимущества

Преимущество продукта не в том, что он просто показывает новости. Новостных лент и поисковиков уже много.

Сильная сторона — в постобработке потока. Аналитик быстрее получает структуру обзора, видит меньше повторов и меньше рискует пропустить значимое обновление.

Ещё одно важное преимущество — редакторский контроль. Модель предлагает метки, но человек может их исправить вручную. Поэтому продукт не превращается в чёрный ящик.

Итоговая идея простая: сервис не просто ищет новости — он превращает поток в рабочую структуру.

Слайд 12. Итоги и планы

Я честно разделяю техническую и продуктовую проверку.

В рамках проекта я проверил техническую гипотезу: можно ли собрать новости в сюжеты, выделить значимые обновления и обернуть это в работающий сервис.

Основные риски тоже понятны. Во-первых, пока малый объём ручной разметки. Во-вторых, ещё нет пилота с клиентом. В-третьих, нужно учитывать конкуренцию с LLM-инструментами и крупными SaaS-решениями.

Следующие шаги — расширить ручную разметку, доработать ML-пайплайн, добавить пакетную обработку больших импортов, LLM-названия сюжетов и больше форматов интеграции, например RSS и API.

Отдельный продуктовый шаг — пилот на реальном новостном потоке, чтобы проверить ценность и монетизацию. Гипотеза монетизации — B2B-подписка или лицензия.

На этом всё. Спасибо, готов ответить на вопросы.